

Technical Requirements Document (TRD)

Project Name: AI Store

1. Introduction

This document outlines the technical architecture, database design, API specifications, and deployment strategy for the AI Tools Marketplace. It serves as the primary technical reference for developers to ensure the platform is built securely, scales efficiently, and meets the requirements defined in the PRD.

2. System Architecture

The platform will utilize a modern decoupled architecture:

- **Client (Frontend):** Next.js application handling UI rendering, routing, and state management. Utilizes Server-Side Rendering (SSR) for dynamic content (Leaderboard, Tool Details) and Static Site Generation (SSG) for static pages.
- **API Layer (Backend):** Node.js serving as the backend layer (either via Next.js API Routes or a standalone Express/Node server) to process business logic, handle complex queries, and format data.
- **Data & Auth Layer:** Supabase providing PostgreSQL database hosting, Row Level Security (RLS), and authentication for the Admin Dashboard.

3. Technology Stack

- **Frontend Framework:** Next.js (App Router recommended), React
- **Styling:** Tailwind CSS, PostCSS, HTML5/CSS3
- **Language:** TypeScript (Strict mode enabled)
- **Backend Runtime:** Node.js
- **Database:** PostgreSQL (Hosted on Supabase)
- **Authentication:** Supabase Auth (Email/Password for Admins)
- **ORM / Query Builder:** Prisma or Drizzle ORM (Recommended for type-safe DB interactions)
- **Content Parsing:** `next-mdx-remote` or `react-markdown` (for rendering Markdown tutorials)
- **Deployment:** Vercel (Frontend & Serverless APIs), Supabase (Database/Auth)
- **Version Control:** Git & GitHub

4. Database Schema (PostgreSQL)

4.1 Tables

1. categories

- `id` (UUID/Serial, PK)
- `name` (String, Unique)
- `slug` (String, Unique) - For SEO-friendly URLs
- `description` (Text)

2. tools

- `id` (UUID/Serial, PK)
- `name` (String)
- `slug` (String, Unique)
- `description` (Text)
- `website_url` (String)
- `pricing_model` (Enum: 'Free', 'Freemium', 'Paid', 'Usage-Based')
- `capability_score` (Numeric 3,1)
- `speed_score` (Numeric 3,1)
- `cost_efficiency_score` (Numeric 3,1)
- `usefulness_score` (Numeric 3,1)
- `overall_score` (Numeric 3,1) - *Calculated field or trigger-updated*
- `created_at` (Timestamp)

3. tool_categories (Join Table)

- `tool_id` (FK to tools.id)
- `category_id` (FK to categories.id)
- *Primary Key: (tool_id, category_id)*

4. tutorials

- `id` (UUID/Serial, PK)
- `tool_id` (FK to tools.id)
- `title` (String)
- `content` (Text - Markdown)
- `prompt_strategies` (Text - Markdown)
- `token_optimization_tips` (Text - Markdown)

- `created_at` (Timestamp)
- `updated_at` (Timestamp)

5. API Design (Node.js / Next.js API Routes)

All endpoints will return JSON responses. Admin routes will require a valid Supabase JWT Bearer token.

5.1 Public Endpoints

- `GET /api/tools` - Fetch a paginated list of all tools.
 - *Query Params:* `?category={slug}`, `?search={query}`, `?pricing={model}`
- `GET /api/tools/leaderboard` - Fetch tools sorted by `overall_score` descending.
- `GET /api/tools/:slug` - Fetch details for a specific tool, including joined categories.
- `GET /api/tools/:slug/tutorials` - Fetch tutorials associated with a specific tool.
- `GET /api/categories` - Fetch all available categories.

5.2 Protected Admin Endpoints (Requires Auth)

- `POST /api/admin/tools` - Create a new tool listing.
- `PUT /api/admin/tools/:id` - Update an existing tool.
- `DELETE /api/admin/tools/:id` - Remove a tool.
- `POST /api/admin/tutorials` - Create a new tutorial.
- `PUT /api/admin/tutorials/:id` - Update an existing tutorial.

6. Frontend Architecture (Next.js)

6.1 Recommended Folder Structure

```

/
├── app/                                # Next.js App Router
│   ├── (public)/                       # Public facing pages
│   │   ├── page.tsx                   # Home/Directory page
│   │   ├── leaderboard/               # Leaderboard page
│   │   ├── category/[slug]/           # Filtered category views
│   │   └── tool/[slug]/               # Tool detail & tutorial page
│   ├── (admin)/                       # Protected admin dashboard
│   │   ├── dashboard/                 # Main admin panel
│   │   ├── login/                    # Admin login
│   │   └── layout.tsx                 # Admin layout with auth wrappers
│   ├── api/                           # Backend API Routes (Node.js logic)
│   ├── layout.tsx                     # Root layout (Header, Footer)
│   └── globals.css                    # Tailwind injections
└── components/                        # Reusable UI components

```

├── ui/	# Buttons, Inputs, Cards
├── tools/	# ToolCard, ToolGrid, LeaderboardTable
├── admin/	# MarkdownEditor, ToolForm
├── lib/	# Utility functions
├── db.ts	# Database connection/ORM config
├── supabase.ts	# Supabase client initialization
├── utils.ts	# Helper functions (formatting, classes)
├── types/	# TypeScript interfaces
└── index.d.ts	# Types for Tools, Categories, etc.

6.2 State Management

- **Server State:** Handled natively by Next.js React Server Components (RSC) and standard `fetch` caching mechanisms.
- **Client State:** React `useState` and `useContext` for local UI interactions (e.g., mobile menu toggle, search input state, active filters).

7. Security & Authentication

- **Authentication:** Managed via Supabase Auth. Only approved admin emails will be granted access to the dashboard.
- **Authorization:** Middleware in Next.js will intercept requests to `/dashboard` and `/api/admin/*` to verify the presence and validity of the user's JWT.
- **Database Security:** Supabase Row Level Security (RLS) policies will be configured to:
 - Allow `SELECT` operations to `anon` (public) users.
 - Restrict `INSERT`, `UPDATE`, `DELETE` operations to `authenticated` admin users only.
- **Input Validation:** Implement `Zod` or `Yup` for rigorous server-side validation of all incoming data on API routes to prevent SQL injection and malformed data.

8. Deployment Strategy

- **Frontend & API Routes:** Deployed automatically to Vercel upon merging into the `main` branch on GitHub. Environment variables (Supabase URL, Anon Key, Service Role Key) will be securely stored in Vercel settings.
- **Database:** Hosted on Supabase. Schema migrations will be tracked and executed using a tool like Prisma CLI or Supabase CLI.
- **CI/CD:** GitHub Actions (optional but recommended) to run TypeScript type checking and linting before allowing merges to the main deployment branch.